Intro to Linear Regression + Model

এতক্ষণ আমরা AI, ML, এবং Pipeline নিয়ে কথা বলেছি। এখন থেকে আমরা আমাদের প্রথম actual ML algorithm শিখবো — Linear Regression। এটা ML-এর সবচেয়ে fundamental algorithm। এটা বুঝলে পরের সব complex algorithm বুঝতে অনেক সহজ হবে।

আমরা কী সমস্যা solve করতে চাই?

ধরো তোমার কাছে কিছু data আছে:

Experience (বছর)	Salary (হাজার টাকা)
1	15
3	35
4	45
5	65
6	75

এখন কেউ জিজ্ঞেস করলো — 5 বছর experience আছে এমন কারো salary কত হবে? এই data-তে 5 নেই। কিন্তু pattern দেখলেই বোঝা যাচ্ছে — experience বাড়লে salary বাড়ছে। Linear Regression এই pattern টাকে mathematically capture করে এবং নতুন value predict করে।

Linear Regression-এর মূল idea

Data-র মধ্য দিয়ে একটা সরলরেখা আঁকো যেটা সব points-এর সবচেয়ে কাছে থাকে। এই line-টাই হলো আমাদের model। নতুন কোনো input দিলে এই line থেকে output predict করা যাবে।
এই line-কে mathematically লেখা হয়:

$$\hat{y} = wx + b$$

যেখানে:

- **x** = input (যেটা আমরা জানি, যেমন experience)
- **ŷ** = predicted output (যেটা জানতে চাই, যেমন salary)
- **w** = weight (line-এর slope — কতটা খাড়া)
- **b** = bias (line টা y-axis-এ কোথায় শুরু হয়)

w এবং b মানে কী?

এই দুটো জিনিস বোঝাটা অনেক গুরুত্বপূর্ণ।
w (weight/slope) নিয়ন্ত্রণ করে line কতটা খাড়া হবে। w বড় হলে line বেশি খাড়া — মানে experience একটু বাড়লেই salary অনেক বাড়বে। w ছোট হলে line অনেক flat।
b (bias/intercept) নিয়ন্ত্রণ করে line টা কোথা থেকে শুরু হবে। b মানে হলো যদি experience 0 বছর হয়, তাহলেও যে base salary থাকে।

এই দুটো মিলিয়ে line-এর পুরো shape নির্ধারিত হয়।

Code দিয়ে দেখি

```
import numpy as np

# আমাদের data
X = np.array([1, 3, 4, 6, 7])   # Experience (বছর)
y = np.array([15, 35, 45, 65, 75])  # Salary (হাজার)

# Prediction function
def make_prediction(X, w, b):
    m = X.shape[0]              # কতগুলো data point আছে
    predictions = np.zeros(m)   # empty array তৈরি

    for i in range(m):
        predictions[i] = w * X[i] + b  # ŷ = wx + b

    return predictions
```

Parameters Experiment করি

এখন বিভিন্ন w এবং b দিয়ে দেখি কী হয়:

w	b	Prediction (x=5)	কেমন fit?
0	0	0	খুবই খারাপ — flat line
1	0	5	অনেক নিচে
5	3	28	কাছাকাছি কিন্তু কম
10	5	55	অনেক ভালো fit

w=10, b=5 দিলে prediction হয়:

```
x=1 → 10×1 + 5 = 15  ✓  (actual: 15)
x=3 → 10×3 + 5 = 35  ✓  (actual: 35)
x=4 → 10×4 + 5 = 45  ✓  (actual: 45)
```

```
x=6 → 10×6 + 5 = 65  ✓  (actual: 65)
x=7 → 10×7 + 5 = 75  ✓  (actual: 75)
```

Perfect fit! কিন্তু এখানে একটা বড় প্রশ্ন আসে।

সমস্যাটা কোথায়?

আমরা manually try করে w=10, b=5 খুঁজে পেলাম। কিন্তু real world-এ হাজার হাজার data point থাকে। সেখানে manually try করা সম্ভব না।

তাহলে machine কীভাবে automatically সঠিক w এবং b খুঁজে বের করবে?

এই প্রশ্নের উত্তর হলো **Cost Function** এবং **Gradient Descent** — যেটা আমরা পরের page-এ শিখবো।

Model মানে আসলে কী?

এই পুরো আলোচনায় একটা গুরুত্বপূর্ণ concept clear হওয়া দরকার।

Linear Regression-এ **model** মানে হলো শুধু এই দুটো number — w এবং b। এই দুটো ঠিক হয়ে গেলে যেকোনো নতুন input-এ prediction করা যাবে।

```
Model = (w, b) = (10, 5)

নতুন input: x = 10 বছর experience
Prediction: 10×10 + 5 = 105 হাজার টাকা
```

Machine Learning মানে আসলে এই w এবং b-এর সঠিক value automatically খুঁজে বের করা — data থেকে।

Summary

Linear Regression হলো ML-এর সবচেয়ে fundamental algorithm যেখানে data-র মধ্য দিয়ে একটা best-fit line আঁকা হয়। এই line-এর equation হলো ŷ = wx + b, যেখানে w হলো slope এবং b হলো intercept। Model মানে হলো এই w এবং b-এর সঠিক value। কিন্তু manually এই value খোঁজা সম্ভব না — এজন্য দরকার Cost Function এবং Gradient Descent।

Parameters + Cost Function

আগের page-এ আমরা দেখেছি Linear Regression-এ model মানে হলো w এবং b খুঁজে বের করা। কিন্তু manually try করে সঠিক w এবং b পাওয়া real world-এ সম্ভব না। তাহলে machine কীভাবে বুঝবে কোন w এবং b সবচেয়ে ভালো? এই প্রশ্নের উত্তর হলো Cost Function।

Parameters মানে কী?

Linear Regression-এ w এবং b কে বলা হয় **parameters**। এই parameters-ই model-এর সব কিছু নিয়ন্ত্রণ করে।

```
Parameter  |  নাম               |  ভূমিকা
-----------|--------------------|----------------------------
w          |  Weight / Slope    |  Line কতটা খাড়া হবে
b          |  Bias / Intercept  |  Line কোথায় শুরু হবে
```

Machine Learning-এর মূল কাজ হলো data দেখে এই parameters-এর সঠিক value automatically খুঁজে বের করা।

সমস্যাটা কী?

ধরো আমাদের data:

```
Experience (x)  |  Actual Salary (y)
----------------|-------------------
1               |  15
3               |  35
4               |  45
6               |  65
7               |  75
```

আমরা যদি w=5, b=0 দিই, তাহলে predictions হবে:

```
x=1 → 5×1 + 0 = 5    কিন্তু actual = 15   (error = 10)
x=3 → 5×3 + 0 = 15   কিন্তু actual = 35   (error = 20)
x=4 → 5×4 + 0 = 20   কিন্তু actual = 45   (error = 25)
```

Prediction এবং actual value-এর মধ্যে পার্থক্য আছে। এই পার্থক্যকে বলে **error**। Machine-কে এই error কমাতে হবে। কিন্তু "কতটা error আছে" সেটা measure করতে হলে একটা formula দরকার — সেটাই Cost Function।

Cost Function কী?

Cost Function হলো একটা formula যেটা বলে দেয় আমাদের current w এবং b দিয়ে model কতটা ভুল করছে। Cost যত বেশি, model তত খারাপ। Cost যত কম, model তত ভালো।

সবচেয়ে common Cost Function হলো **Mean Squared Error (MSE)**:

$$J(w,b) = \frac{1}{2m} \times \Sigma(\hat{y}_i - y_i)^2$$

```
Symbol   |  অর্থ
---------|-----------------------------------------
J(w,b)   |  Cost Function (এটা minimize করতে হবে)
m        |  Training examples-এর সংখ্যা
```

```
ŷᵢ       |  Model-এর prediction
yᵢ       |  Actual value
(ŷ - y)  |  Error (prediction এবং actual-এর পার্থক্য)
```

Error কে square করা হয় দুটো কারণে। প্রথমত, negative এবং positive error একে অপরকে cancel করে দিতে পারে — square করলে সব positive হয়। দ্বিতীয়ত, বড় error-কে আরো বেশি penalize করা যায়।

হাতে কলমে Calculate করি

w=5, b=0 দিয়ে cost calculate করি:

x	Prediction	Actual	Error	Error²
1	5	15	-10	100
3	15	35	-20	400
4	20	45	-25	625
6	30	65	-35	1225
7	35	75	-40	1600

Total error² = 100+400+625+1225+1600 = 3950

Cost J = (1/2×5) × 3950 = 395

এখন w=10, b=5 দিয়ে দেখি:

x	Prediction	Actual	Error	Error²
1	15	15	0	0
3	35	35	0	0
4	45	45	0	0
6	65	65	0	0
7	75	75	0	0

Total error² = 0

Cost J = 0

Cost 395 থেকে 0 হয়ে গেছে। মানে w=10, b=5 অনেক ভালো parameters।

Code-এ Cost Function

```
def compute_cost(X, y, w, b):
    """
```

```
    MSE Cost Function calculate করে
    X: input features
    y: actual values
    w: weight, b: bias
    """
    m = X.shape[0]      # training examples-এর সংখ্যা
    total_cost = 0

    for i in range(m):
        prediction = w * X[i] + b       # ŷ = wx + b
        error = prediction - y[i]       # (ŷ - y)
        total_cost += error ** 2        # error²

    cost = (1 / (2 * m)) * total_cost  # (1/2m) × Σ error²
    return cost

# Test করি
X = np.array([1, 3, 4, 6, 7])
y = np.array([15, 35, 45, 65, 75])

print(compute_cost(X, y, w=5,  b=0))   # Output: 395.0
print(compute_cost(X, y, w=10, b=5))   # Output: 0.0
```

Cost Function কীভাবে help করে?

Cost Function এককথায় একটা scoreboard। w এবং b-এর প্রতিটি combination-এ একটা cost আছে। Machine-এর কাজ হলো সেই combination খুঁজে বের করা যেখানে cost সবচেয়ে কম।

```
w    |  b  |  Cost
-----|-----|-------
0    |  0  |  অনেক বেশি
5    |  0  |  395
8    |  3  |  মাঝামাঝি
10   |  5  |  0  ← এটাই চাই
```

এই "cost কমানোর" কাজটা করে **Gradient Descent** — যেটা পরের module-এ শিখবো।

Summary

Parameters হলো w এবং b — এই দুটো value-ই Linear Regression model-এর সব কিছু নির্ধারণ করে। Cost Function হলো MSE formula যেটা measure করে model কতটা ভুল করছে। Cost যত কম, model তত ভালো। Machine Learning-এ model train করা মানে আসলে এই Cost Function-কে minimize করার মতো w এবং b খুঁজে বের করা। পরের module-এ দেখবো Gradient Descent কীভাবে automatically এই কাজটা করে।

Implementation of Cost Function

এই মডিউলে আমরা শিখব কীভাবে Linear Regression মডেলে Cost Function ইমপ্লিমেন্ট করা হয় এবং কেন এটি গুরুত্বপূর্ণ। আমরা আগের মডিউলে তৈরি করা Prediction Function ব্যবহার করে Model-এর Error পরিমাপ করব।

Cost Function কী?

Machine Learning-এর মূল লক্ষ্য হলো এমন একটি সরলরেখা (Line) খুঁজে বের করা যা আমাদের ডেটার সাথে সবচেয়ে ভালোভাবে মানিয়ে যায়।

আমাদের Linear Regression মডেল:

f(x) = wx + b

এখানে,

w = Weight (Slope)
b = Bias (Intercept)
x = Input Feature
f(x) = Prediction

কিন্তু একটি প্রশ্ন আসে:

"আমাদের Prediction কতটা সঠিক?"

এই প্রশ্নের উত্তর দেওয়ার জন্য আমরা Cost Function ব্যবহার করি।

Cost Function Model-এর Prediction এবং Actual Value-এর মধ্যকার পার্থক্য পরিমাপ করে।

আমাদের **Dataset**

```
1X = np.array([1, 3, 4, 6, 7])
```

```
2y = np.array([15, 35, 45, 65, 75])
```

এখানে,

Experience (X)	Salary (y)
1	15

3	35
4	45
6	65
7	75

Error কী?

ধরুন,

```
w = 5
```

```
b = 3
```

তাহলে Prediction হবে:

```
prediction = 5 * X + 3
```

প্রথম Data Point-এর জন্য:

```
Prediction = 5 × 1 + 3
           = 8
```

কিন্তু Actual Value:

```
15
```

তাহলে Error:

```
Error = Prediction - Actual
      = 8 - 15
      = -7
```

Error Positive অথবা Negative হতে পারে।

কেন **Error Squared** করা হয়?

যদি আমরা সব Error সরাসরি যোগ করি, তাহলে Positive এবং Negative Error একে অপরকে বাতিল করে দিতে পারে।

উদাহরণ:

```
Error 1 = +10
Error 2 = -10

Total Error = 0
```

কিন্তু বাস্তবে Model-এর ভুল তো আছে।

এই সমস্যা সমাধানের জন্য Error-এর Square নেওয়া হয়।

```
(+10)² = 100
(-10)² = 100
```

এখন Total Error:

```
100 + 100 = 200
```

অর্থাৎ বড় ভুলগুলোও বেশি গুরুত্ব পায়।

Mean Squared Error (MSE)

Linear Regression-এ আমরা সাধারণত নিচের Cost Function ব্যবহার করি:

$$J(w,b) = (1 / 2m) \times \Sigma (f(x_i) - y_i)^2$$

এখানে,

J(w,b) = Cost
m = মোট Data Point সংখ্যা
$f(x_i)$ = Prediction
y_i = Actual Value
Σ = সব Error Squared-এর যোগফল

কেন **2m** দিয়ে ভাগ করা হয়**?**

Formula-টি লেখা হয়:

$$J(w,b) = (1 / 2m) \times \Sigma (f(x_i) - y_i)^2$$

এখানে 2 দিয়ে ভাগ করার কারণ হলো ভবিষ্যতে Gradient Descent-এর Derivative হিসাবকে সহজ করা।

এটি Cost-এর আচরণ পরিবর্তন করে না, শুধুমাত্র গণনাকে সহজ করে।

Cost Function Implementation

এখন আমরা Python-এ Cost Function লিখব।

```python
def compute_cost(X, y, w, b):
    m = X.shape[0]
    cost = 0.0

    for i in range(m):
        prediction = w * X[i] + b

        error = prediction - y[i]

        error_squared = error ** 2

        cost = cost + error_squared

    cost = cost / (2 * m)
```

```
15
16     return cost
```

Code Breakdown

প্রথমে Data Point-এর সংখ্যা বের করা হচ্ছে।

```
1m = X.shape[0]
```

আমাদের Dataset-এ:

```
1m = 5
```

Cost Variable Initialize করা হচ্ছে।

```
1cost = 0.0
```

প্রতিটি Data Point-এর জন্য Loop চালানো হচ্ছে।

```
1for i in range(m):
```

Prediction হিসাব করা হচ্ছে।

```
1prediction = w * X[i] + b
```

Prediction এবং Actual Value-এর পার্থক্য বের করা হচ্ছে।

```
1error = prediction - y[i]
```

Error-এর Square নেওয়া হচ্ছে।

```
1error_squared = error ** 2
```

সব Squared Error যোগ করা হচ্ছে।

```
1cost = cost + error_squared
```

শেষে Average Error বের করা হচ্ছে।

```
1cost = cost / (2 * m)
```

ম্যানুয়াল **Calculation Example**

ধরি,

```
1w = 0
```

```
2b = 0
```

তাহলে সব Prediction হবে:

```
1[0, 0, 0, 0, 0]
```

এখন Error:

```
10 - 15 = -15
20 - 35 = -35
30 - 45 = -45
40 - 65 = -65
50 - 75 = -75
```

Squared Error:

```
1225
21225
32025
44225
55625
```

Total:

```
1225 + 1225 + 2025 + 4225 + 5625
2= 13325
```

Cost:

```
113325 / (2 × 5)
2
3= 13325 / 10
4
5= 1332.5
```

অর্থাৎ,

```
1compute_cost(X, y, 0, 0)
```

ফলাফল হবে:

```
11332.5
```

Cost-এর মান কী বোঝায়**?**

Cost বড় হলে:

```
1Prediction অনেক ভুল
```

Cost ছোট হলে:

```
1Prediction অনেক ভালো
```

Cost শূন্য হলে:

```
Prediction এবং Actual Value সম্পূর্ণ এক
```

অর্থাৎ,

```
ছোট Cost = ভালো Model
```

```
বড় Cost = খারাপ Model
```

বিভিন্ন **Parameter**-এর জন্য **Cost**

```
compute_cost(X, y, 0, 0)
Cost = 1332.5
compute_cost(X, y, 1, 0)
```

```
Cost = অনেক বড়
```

কারণ Model এখনও Data-এর কাছাকাছি যায়নি।

```
compute_cost(X, y, 10, 5)
```

এই ক্ষেত্রে Prediction হবে:

```
[15, 35, 45, 65, 75]
```

যা Actual Value-এর সাথে পুরোপুরি মিলে যায়।
তাই,

```
Cost = 0
```

সংক্ষেপে
এই মডিউলে আমরা শিখলাম:

- Cost Function Model-এর Error পরিমাপ করে।
- Error = Prediction − Actual Value।
- Negative Error-এর সমস্যা এড়াতে Error Squared করা হয়।
- Linear Regression-এ Mean Squared Error (MSE) ব্যবহার করা হয়।
- Cost যত কম হবে Model তত ভালো হবে।
- Cost Function-এর মাধ্যমে আমরা বুঝতে পারি Model কতটা সঠিক Prediction করছে।
- ভবিষ্যতে Gradient Descent ব্যবহার করে আমরা এই Cost-কে ধীরে ধীরে কমিয়ে সর্বনিম্ন অবস্থায় নিয়ে যাব।

Idea of Minimizing the Cost Function

আগের মডিউলে আমরা Cost Function সম্পর্কে শিখেছি এবং দেখেছি কীভাবে এটি Model-এর Prediction Error পরিমাপ করে।
এখন একটি গুরুত্বপূর্ণ প্রশ্ন আসে:
"Cost Function হিসাব করলাম, কিন্তু এর উদ্দেশ্য কী?"
Machine Learning-এর মূল লক্ষ্য শুধু Cost বের করা নয়, বরং Cost-কে যত সম্ভব কমিয়ে আনা। এই মডিউলে আমরা বুঝব কেন Cost কমানো দরকার এবং কীভাবে আমরা সঠিক Weight (w) ও Bias (b) খুঁজে বের করার চেষ্টা করি।

কেন **Cost Minimize** করতে হবে?

আমাদের Linear Regression Model হলো:

```
f(x) = wx + b
```

এখানে w এবং b-এর বিভিন্ন মান ব্যবহার করলে বিভিন্ন ধরনের সরলরেখা (Line) পাওয়া যায়।
কিছু Line ডেটার সাথে ভালোভাবে মিলে যায়, আবার কিছু Line একেবারেই মিলে না।
যখন Line ডেটার কাছাকাছি থাকে:

```
Prediction Error কম হয়
Cost কম হয়
```

যখন Line ডেটা থেকে দূরে থাকে:

```
Prediction Error বেশি হয়
Cost বেশি হয়
```

তাই আমাদের লক্ষ্য হলো এমন একটি Line খুঁজে বের করা যার Cost সবচেয়ে কম।
একটি উদাহরণ
আমাদের Dataset:

```
X = np.array([1, 3, 4, 6, 7])
y = np.array([15, 35, 45, 65, 75])
```

ধরি,

```
w = 0
b = 0
```

তাহলে Prediction হবে:

```
[0, 0, 0, 0, 0]
```

যা Actual Value-এর থেকে অনেক দূরে।
ফলে:

```
Cost = অনেক বড়
```

এবার ধরি,

```
w = 5


b = 3
```

Prediction:

```
[8, 18, 23, 33, 38]
```

এগুলো Actual Value-এর কিছুটা কাছাকাছি এসেছে।
ফলে:

```
Cost আগের চেয়ে কম
```

এবার ধরি,

```
w = 10
```

```
b = 5
```

Prediction:

```
[15, 35, 45, 65, 75]
```

যা Actual Value-এর সাথে পুরোপুরি মিলে গেছে।
তাই:

```
Cost = 0
```

এটাই আমাদের আদর্শ পরিস্থিতি।
Cost Function-কে একটি **Landscape** হিসেবে কল্পনা করা
Cost Function-কে আমরা একটি পাহাড়ি এলাকার মতো কল্পনা করতে পারি।
ধরি,

```
উঁচু স্থান = বড় Cost
```

```
নিচু স্থান = ছোট Cost
```

আমাদের কাজ হলো পাহাড়ের চূড়া থেকে নিচের সর্বনিম্ন স্থানে পৌঁছানো।
সর্বনিম্ন স্থানটি হলো:

```
Minimum Cost
```

এবং সেই স্থানে থাকা w ও b-ই হবে আমাদের Best Parameters।
Cost Function Graph-এর ধারণা
আগের মডিউলে আমরা বিভিন্ন w-এর জন্য Cost বের করেছিলাম।

```
for i in range(-100, 100):
    cost_i = compute_cost(X, y, w=i, b=0)
```

এরপর Cost Plot করেছিলাম।

```
plt.plot(w_range, cost_history)
```

এই Graph সাধারণত U-Shape হয়।

```
Cost
 ^
 |
 |          *
 |       *     *
 |     *         *
 |   *             *
 | *                 *
 +---------------------> w
```

Graph-এর একদম নিচের অংশ নির্দেশ করে:

```
Minimum Cost
```

অর্থাৎ সেই w-এর মান যেখানে Model সবচেয়ে ভালো কাজ করছে।

Minimum Cost মানে কী?

Minimum Cost মানে সবসময় Cost = 0 হবে এমন নয়।

বাস্তব Dataset-এ সাধারণত কিছু Noise থাকে।

ফলে:

```
Prediction পুরোপুরি মিলে না
```

তবুও এমন একটি w এবং b পাওয়া যায় যেখানে Cost সবচেয়ে কম হয়।

আমাদের লক্ষ্য হলো:

```
Possible Lowest Cost খুঁজে বের করা
```

Brute Force Approach

একটি সহজ উপায় হতে পারে:

```
w = -100
w = -99
w = -98
...
w = 100
```

সবগুলোর Cost বের করা।

তারপর যেটার Cost সবচেয়ে কম সেটি নির্বাচন করা।

উদাহরণ:

```
best_cost = float('inf')
best_w = 0

for w in range(-100, 101):
```

```
    cost = compute_cost(X, y, w, 0)

    if cost < best_cost:
        best_cost = cost
        best_w = w
```

এভাবে Minimum Cost খুঁজে পাওয়া সম্ভব।

Brute Force Approach-এর সমস্যা

ছোট Dataset-এর জন্য এটি কাজ করতে পারে।

কিন্তু বাস্তবে:

```
w-এর অসংখ্য সম্ভাব্য মান আছে
b-এরও অসংখ্য সম্ভাব্য মান আছে
```

যদি আমরা প্রতিটি সম্ভাবনা পরীক্ষা করতে যাই:

```
গণনার পরিমাণ অনেক বেড়ে যাবে
সময় অনেক বেশি লাগবে
```

বড় Dataset-এর ক্ষেত্রে এটি কার্যকর নয়।

তাহলে কী করা হবে**?**

আমাদের এমন একটি পদ্ধতি দরকার যা:

```
সরাসরি Minimum Cost-এর দিকে এগোবে
অপ্রয়োজনীয় মান পরীক্ষা করবে না
দ্রুত Best Parameters খুঁজে পাবে
```

Machine Learning-এ এই কাজের জন্য ব্যবহার করা হয়:

```
Gradient Descent
```

Gradient Descent ধীরে ধীরে w এবং b আপডেট করে Cost কমাতে থাকে, যতক্ষণ না আমরা Minimum Cost-এর কাছাকাছি পৌঁছে যাই।

এই মডিউলের মূল ধারণা

এই মডিউলে আমরা শিখলাম:

Cost Function-এর উদ্দেশ্য হলো Prediction Error পরিমাপ করা।
একটি ভালো Model-এর Cost কম হয়।
আমাদের লক্ষ্য হলো Cost-কে সর্বনিম্ন পর্যায়ে নিয়ে আসা।
বিভিন্ন w এবং b বিভিন্ন Cost তৈরি করে।
Cost Function Graph সাধারণত U-Shape হয়।
Graph-এর সর্বনিম্ন বিন্দুতে Minimum Cost পাওয়া যায়।
Brute Force পদ্ধতিতে Minimum Cost খুঁজে পাওয়া গেলেও এটি কার্যকর নয়।
তাই আমরা Gradient Descent ব্যবহার করি, যা দ্রুত এবং কার্যকরভাবে Minimum Cost-এর দিকে এগিয়ে যায়।

Idea of Gradient Descent

আগের মডিউলে আমরা দেখেছি যে Machine Learning-এর মূল লক্ষ্য হলো Cost Function-কে যতটা সম্ভব কমিয়ে আনা। আমরা এটাও বুঝেছি যে বিভিন্ন Weight (w) এবং Bias (b) বিভিন্ন Cost তৈরি করে, এবং আমাদের এমন একটি w ও b খুঁজে বের করতে হবে যেখানে Cost সর্বনিম্ন হয়।
কিন্তু প্রশ্ন হলো:

```
1কীভাবে আমরা সেই Minimum Cost-এর দিকে পৌঁছাব?
```

এই সমস্যার সমাধানই হলো **Gradient Descent**।
Gradient Descent কী?
Gradient Descent হলো একটি Optimization Algorithm।
এর কাজ হলো:

```
1বর্তমান অবস্থান থেকে ধাপে ধাপে এমন দিকে এগোনো,
```

```
2যেখানে Cost কমতে থাকে।
```

অর্থাৎ এটি Weight (w) এবং Bias (b) বারবার পরিবর্তন করে Cost কমানোর চেষ্টা করে।
একটি পাহাড়ের উদাহরণ
ধরুন আপনি একটি পাহাড়ের ঢালে দাঁড়িয়ে আছেন।
আপনার লক্ষ্য:

```
1পাহাড়ের সর্বনিম্ন স্থানে পৌঁছানো
```

কিন্তু সমস্যা হলো:

```
1চারপাশে ঘন কুয়াশা
```

তাই আপনি একসাথে পুরো পথ দেখতে পাচ্ছেন না।
তাহলে আপনি কী করবেন?
আপনি চারপাশ দেখে বুঝতে চেষ্টা করবেন:

```
1কোন দিকে নামলে নিচের দিকে যাওয়া যায়
```

তারপর ছোট একটি পদক্ষেপ নেবেন।
নতুন অবস্থানে গিয়ে আবার দেখবেন:

```
1এখন কোন দিকে নামা উচিত?
```

এভাবেই ধাপে ধাপে নিচে পৌঁছে যাবেন।
Gradient Descent ঠিক এই কাজটিই করে।
Cost Function-এর ক্ষেত্রে পাহাড়টি কী**?**

Gradient Descent-এ:

```
1পাহাড়ের উচ্চতা = Cost
```

অর্থাৎ,

```
1উঁচু স্থান = বড় Cost
```

```
2নিচু স্থান = ছোট Cost
```

আমাদের লক্ষ্য:

```
1Cost-এর সর্বনিম্ন বিন্দুতে পৌঁছানো
```

Gradient Descent-এর মূল ধারণা
ধরি,
বর্তমানে আমাদের Weight:

```
1w = 0
```

এবং Cost:

```
11332.5
```

এখন আমরা জানতে চাই:

```
1w বাড়ালে Cost বাড়বে নাকি কমবে?
```

যদি Cost কমে:

```
1সেদিকেই এগোব
```

যদি Cost বাড়ে:

```
1উল্টো দিকে যাব
```

এই সিদ্ধান্ত নেওয়ার জন্য Gradient ব্যবহার করা হয়।
Gradient কী বলে**?**
Gradient আমাদের জানায়:

```
1কোন দিকে গেলে Cost দ্রুত বাড়ে
```

কিন্তু আমাদের লক্ষ্য Cost কমানো।
তাই আমরা Gradient-এর বিপরীত দিকে এগোই।

```
1Gradient ↑
```

```
2আমরা যাই ↓
```

এ কারণেই নাম:

```
1Gradient Descent
```

অর্থাৎ Gradient-এর বিপরীত দিকে নেমে যাওয়া।
উদাহরণ
ধরি Cost Function-এর একটি অংশ এমন:

```
1          Cost
2
3             *
4           *
5         *
6       *
7     *
8   *
9--------------------> w
```

এখানে আমরা ডানদিকে গেলে Cost বাড়ছে।
অর্থাৎ:

```
1Gradient Positive
```

তাই Gradient Descent বলবে:

```
1বামদিকে যাও
```

কারণ বামদিকে গেলে Cost কমবে।
আবার যদি Graph এমন হয়:

```
1Cost
2
3*
4  *
5    *
6      *
7        *
8          *
9--------------------> w
```

এখানে ডানদিকে গেলে Cost কমছে।
অর্থাৎ:

```
1Gradient Negative
```

তাই Gradient Descent বলবে:

```
1ডানদিকে যাও
```

Gradient Descent-এর **Step-by-Step** চিন্তা
ধরি আমরা শুরু করেছি:

```
1w = 0
```

এখন Cost অনেক বড়।
Gradient Descent করবে:

```
1Step 1:

2Cost কোন দিকে কমছে খুঁজে বের করবে
```

তারপর:

```
1Step 2:

2w-কে সামান্য পরিবর্তন করবে
```

এরপর:

```
1Step 3:

2নতুন Cost বের করবে
```

আবার:

```
1Step 4:

2Cost এখনও কমানো যায় কি না দেখবে
```

এভাবে বারবার চলতে থাকবে।
একবারে **Minimum-**এ কেন যাওয়া হয় না**?**
ধরি আপনি পাহাড় থেকে নামছেন।
যদি একবারে বিশাল লাফ দেন:

```
1আপনি হয়তো গন্তব্য পেরিয়ে যাবেন
```

অথবা

```
1ভুল জায়গায় চলে যাবেন
```

তাই Gradient Descent ছোট ছোট পদক্ষেপ নেয়।

```
1ছোট Step
```

```
2→ বেশি নিয়ন্ত্রণ
```

```
3
```

```
4বড় Step
```

```
5→ Overshoot হওয়ার সম্ভাবনা
```

এই Step-এর আকারকে পরে Learning Rate বলা হবে।
উদাহরণ
ধরি:

```
1w = 0
```

```
2Cost = 1332.5
```

Gradient Descent কিছুটা এগিয়ে গেল:

```
1w = 2
```

```
2Cost = 900
```

আরও এগোল:

```
1w = 5
```

```
2Cost = 400
```

আরও এগোল:

```
1w = 8
```

```
2Cost = 50
```

আরও এগোল:

```
1w = 10
```

```
2Cost = 0
```

দেখা যাচ্ছে:

```
1প্রতিটি ধাপে Cost কমছে
```

এটাই Gradient Descent-এর মূল লক্ষ্য।
Minimum Point-এ পৌঁছালে কী হয়**?**
এক সময় এমন একটি অবস্থায় পৌঁছানো যায় যেখানে:

```
1আর কোনো দিকে গেলেও Cost কমে না
```

সেই অবস্থাকে বলা হয়:

```
Minimum Point
```

সেখানে:

```
Gradient ≈ 0
```

কারণ:

```
নিচে নামার মতো আর কোনো দিক নেই
```

ঠিক যেমন পাহাড়ের একদম নিচের স্থানে পৌঁছে গেলে আর নামা যায় না।

Linear Regression-এ Gradient Descent কেন গুরুত্বপূর্ণ?

আমাদের Model:

```
f(x) = wx + b
```

এখানে আমাদের খুঁজতে হবে:

```
Best w
Best b
```

Manual Trial-and-Error করে এটি সম্ভব হলেও বড় Dataset-এর জন্য কার্যকর নয়।

Gradient Descent:

```
স্বয়ংক্রিয়ভাবে
w এবং b আপডেট করে
Cost কমায়
এবং Best Fit Line খুঁজে বের করে
```

Gradient Descent-এর সাধারণ Update Rule

Gradient Descent প্রতিবার Parameter Update করে।

Weight-এর জন্য:

```
new_w = old_w - α × gradient
```

Bias-এর জন্য:

```
new_b = old_b - α × gradient
```

এখানে:

```
α = Learning Rate
```

বর্তমানে Formula মুখস্থ করার দরকার নেই।

এই মডিউলের উদ্দেশ্য শুধু ধারণা বোঝা।

পরবর্তী মডিউলে আমরা Update Process-টি হাতে-কলমে Simulation করে দেখব।

এই মডিউলের মূল ধারণা

Gradient Descent হলো একটি Optimization Algorithm।
এর কাজ হলো Cost Function-কে ধীরে ধীরে কমানো।
এটি পাহাড় থেকে নিচে নামার মতো কাজ করে।
Gradient বলে দেয় Cost কোন দিকে দ্রুত বাড়ছে।
আমরা Gradient-এর বিপরীত দিকে এগোই।
প্রতিটি ধাপে Weight এবং Bias আপডেট করা হয়।
প্রতিটি Update-এর পরে Cost আরও কমে।
Minimum Point-এ পৌঁছালে Gradient প্রায় শূন্য হয়ে যায়।
Linear Regression-এ Best Fit Line খুঁজে বের করার জন্য Gradient Descent ব্যবহার করা হয়।

Update Simulation through Gradient Descent

আগের মডিউলে আমরা Gradient Descent-এর মূল ধারণা শিখেছি। আমরা দেখেছি যে Gradient Descent ধাপে ধাপে Weight (w) এবং Bias (b) পরিবর্তন করে Cost কমানোর চেষ্টা করে।
কিন্তু এখনও একটি গুরুত্বপূর্ণ প্রশ্ন রয়ে গেছে:

```
Gradient Descent আসলে Update কীভাবে করে?
```

এই মডিউলে আমরা তাত্ত্বিকভাবে একটি Simulation-এর মাধ্যমে দেখব কীভাবে প্রতিটি Update-এর পর Model ধীরে ধীরে Minimum Cost-এর দিকে এগিয়ে যায়।
Gradient Descent-এর মূল ধারণা আবার মনে করি
আমাদের লক্ষ্য:

```
Cost কমানো
```

এবং Cost কমানোর জন্য:

```
Weight (w) আপডেট করা
Bias (b) আপডেট করা
```

Gradient Descent প্রতিটি ধাপে বর্তমান অবস্থান থেকে একটু একটু করে Parameter পরিবর্তন করে।
একটি পাহাড়ে নামার উদাহরণ
ধরুন আপনি একটি পাহাড়ের ঢালে দাঁড়িয়ে আছেন।
বর্তমান অবস্থান:

```
উচ্চতা = 100 মিটার
```

আপনার লক্ষ্য:

```
উচ্চতা = 0 মিটার
```

একবারে নিচে লাফ না দিয়ে আপনি ধাপে ধাপে নামবেন।

```
100 → 80 → 60 → 40 → 20 → 0
```

Gradient Descent-ও একই কাজ করে।

```
বড় Cost → ছোট Cost → আরও ছোট Cost → Minimum Cost
```

Simulation শুরু করা যাক

ধরি,

```
w = 0
```

```
b = 0
```

আমাদের Dataset:

```
X = np.array([1, 3, 4, 6, 7])
```

```
y = np.array([15, 35, 45, 65, 75])
```

এই অবস্থায় Prediction:

```
[0, 0, 0, 0, 0]
```

Cost:

```
1332.5
```

এটি একটি খুব খারাপ Model।

প্রথম **Update**

Gradient Descent বুঝতে পারল:

```
বর্তমান w খুব ছোট
```

তাই এটি w-কে সামান্য বাড়াল।

ধরি:

```
w = 2
```

```
b = 0
```

এখন Prediction:

```
[2, 6, 8, 12, 14]
```

Prediction আগের তুলনায় Actual Value-এর কিছুটা কাছাকাছি এসেছে।

ফলে:

```
Cost কমেছে
```

ধরি Cost হলো:

```
1900
```

দ্বিতীয় **Update**
Gradient Descent আবার দেখল:

```
1Cost এখনও অনেক বেশি
```

তাই আবার Update করল।

```
1w = 4


2b = 1
```

Prediction:

```
1[5, 13, 17, 25, 29]
```

এখন Prediction আরও ভালো হয়েছে।
ফলে:

```
1Cost আরও কমেছে
```

ধরি:

```
1Cost = 550
```

তৃতীয় **Update**
আবার Update করা হলো।

```
1w = 6


2b = 2
```

Prediction:

```
1[8, 20, 26, 38, 44]
```

Prediction এখন Actual Value-এর আরও কাছাকাছি।
ফলে:

```
1Cost = 250
```

চতুর্থ **Update**
Gradient Descent থামছে না।
কারণ এখনও Cost শূন্য হয়নি।
নতুন Parameter:

```
w = 8
```

```
b = 4
```

Prediction:

```
[12, 28, 36, 52, 60]
```

Cost:

```
150
```

এখন Model অনেক ভালো।
পঞ্চম **Update**
আরও একটি Update:

```
w = 10
```

```
b = 5
```

Prediction:

```
[15, 35, 45, 65, 75]
```

যা Actual Value-এর সাথে পুরোপুরি মিলে গেছে।
তাই:

```
Cost = 0
```

Gradient Descent তার লক্ষ্য অর্জন করেছে।
Simulation-টি একসাথে দেখলে

```
Iteration 0
w = 0
b = 0
Cost = 1332.5

Iteration 1
w = 2
b = 0
Cost = 900

Iteration 2
w = 4
b = 1
Cost = 550
```

```
15
16Iteration 3
17w = 6
18b = 2
19Cost = 250
20
21Iteration 4
22w = 8
23b = 4
24Cost = 50
25
26Iteration 5
27w = 10
28b = 5
29Cost = 0
```

এখানে আমরা দেখতে পাচ্ছি:

```
1প্রতিটি Iteration-এ Cost কমছে
```

এটাই Gradient Descent-এর সবচেয়ে গুরুত্বপূর্ণ বৈশিষ্ট্য।
কেন একাধিক **Iteration** লাগে**?**
Gradient Descent সাধারণত একবারে Best Parameter খুঁজে পায় না।
কারণ:

```
1প্রথম Update-এর পরও Cost থাকে

2দ্বিতীয় Update-এর পরও Cost থাকে

3তৃতীয় Update-এর পরও Cost থাকে
```

তাই বারবার Update করতে হয়।
এই পুনরাবৃত্ত Update-গুলোকেই বলা হয়:

```
1Iterations
```

Learning Rate-এর ভূমিকা
প্রতিটি Update-এর আকার সমান নয়।
Update কত বড় হবে তা নির্ধারণ করে:

```
1Learning Rate (α)
```

যদি Learning Rate খুব ছোট হয়:

```
1Cost ধীরে ধীরে কমবে
```

```
2অনেক Iteration লাগবে
```

যদি Learning Rate খুব বড় হয়:

```
1Minimum Point অতিক্রম করে যেতে পারে
```

```
2Cost আবার বেড়ে যেতে পারে
```

তাই Learning Rate সঠিকভাবে নির্বাচন করা গুরুত্বপূর্ণ।
বাস্তবে কী ঘটে?
বাস্তব Machine Learning Model-এ আমরা সাধারণত হাজার হাজার Iteration চালাই।
উদাহরণ:

```
1iterations = 1000
```

প্রতিটি Iteration-এ:

```
1Gradient হিসাব করা হয়
```

```
2w Update করা হয়
```

```
3b Update করা হয়
```

```
4Cost হিসাব করা হয়
```

ধীরে ধীরে Model আরও ভালো হতে থাকে।
কখন **Gradient Descent** থামে?
Gradient Descent সাধারণত থামে যখন:

```
1Cost আর উল্লেখযোগ্যভাবে কমে না
```

অথবা

```
1Gradient প্রায় শূন্য হয়ে যায়
```

অথবা

```
1নির্ধারিত Iteration সংখ্যা শেষ হয়
```

বাস্তব ধারণা
Gradient Descent-কে আপনি একজন ছাত্রের শেখার প্রক্রিয়ার সাথে তুলনা করতে পারেন।
প্রথম পরীক্ষায়:

```
1অনেক ভুল
```

তারপর:

```
1ভুল থেকে শেখা
```

পরবর্তী পরীক্ষায়:

```
1কম ভুল
```

আবার শেখা।
আবার পরীক্ষা।
এভাবে ধীরে ধীরে:

```
1ভুল কমতে থাকে
```

Gradient Descent-ও প্রতিটি Iteration-এ তার ভুল কমানোর চেষ্টা করে।
এই মডিউলের মূল ধারণা

> Gradient Descent একবারে সঠিক Parameter খুঁজে পায় না।
> এটি ধাপে ধাপে Weight এবং Bias আপডেট করে।
> প্রতিটি Update-এর পর Cost পুনরায় হিসাব করা হয়।
> Cost কমতে থাকলে Update সঠিক দিকে হচ্ছে।
> প্রতিটি Update-কে একটি Iteration বলা হয়।
> অনেকগুলো Iteration-এর মাধ্যমে Model Minimum Cost-এর দিকে এগিয়ে যায়।
> Learning Rate Update-এর আকার নিয়ন্ত্রণ করে।
> লক্ষ্য হলো এমন Parameter খুঁজে বের করা যেখানে Cost সর্বনিম্ন হয়।

Calculation of Gradient

আগের মডিউলগুলোতে আমরা Cost Function, Cost Minimization এবং Gradient Descent-এর মূল ধারণা সম্পর্কে জেনেছি। আমরা দেখেছি যে Gradient Descent ধাপে ধাপে Weight (w) এবং Bias (b) পরিবর্তন করে Cost কমানোর চেষ্টা করে।
কিন্তু এখন একটি গুরুত্বপূর্ণ প্রশ্ন আসে:

```
1Gradient Descent কীভাবে বুঝে যে w এবং b কতটুকু পরিবর্তন করতে হবে?
```

এই প্রশ্নের উত্তর হলো **Gradient Calculation**।
এই মডিউলে আমরা Gradient কী, কেন এটি দরকার এবং কীভাবে এটি গণনা করা হয় তা তাত্ত্বিকভাবে বুঝব।
Gradient কী**?**

Gradient হলো Cost Function-এর ঢাল (Slope)।
সহজভাবে বললে Gradient আমাদের বলে:

```
1বর্তমান অবস্থান থেকে কোন দিকে গেলে Cost বাড়বে
```

```
2এবং কত দ্রুত বাড়বে
```

Gradient Descent এই তথ্য ব্যবহার করে Cost কমানোর দিকে এগিয়ে যায়।
একটি বাস্তব উদাহরণ
ধরুন আপনি একটি পাহাড়ের ঢালে দাঁড়িয়ে আছেন।
আপনি জানতে চান:

```
1কোন দিকে নামলে সবচেয়ে দ্রুত নিচে নামা যাবে?
```

এটি জানার জন্য আপনাকে ঢালের দিক জানতে হবে।
যদি ঢাল ডানদিকে উপরের দিকে যায়, তাহলে আপনি বামদিকে হাঁটবেন।
যদি ঢাল বামদিকে উপরের দিকে যায়, তাহলে আপনি ডানদিকে হাঁটবেন।
ঠিক একইভাবে Gradient Cost Function-এর ঢালের তথ্য দেয়।
Linear Regression-এ Gradient কেন দরকার?
আমাদের Model:

```
1prediction = w*x + b
```

এবং Cost Function:

```
1J(w,b) = (1/(2*m)) * Sum((prediction - y)^2)
```

এখানে Cost নির্ভর করছে:

```
1w এর উপর
```

```
2b এর উপর
```

তাই Cost কমাতে হলে আমাদের জানতে হবে:

```
1w পরিবর্তন করলে Cost কীভাবে বদলায়
```

```
2b পরিবর্তন করলে Cost কীভাবে বদলায়
```

এই পরিবর্তনের হারই Gradient।
দুই ধরনের **Gradient**
যেহেতু আমাদের দুইটি Parameter আছে:

```
1w = Weight
```

```
2b = Bias
```

তাই আমাদের দুইটি Gradient লাগবে।
Weight-এর জন্য:

```
1dj_dw
```

Bias-এর জন্য:

```
1dj_db
```

এখানে,

```
1dj_dw = w পরিবর্তন করলে Cost কতটা পরিবর্তিত হয়
```

```
2
```

```
3dj_db = b পরিবর্তন করলে Cost কতটা পরিবর্তিত হয়
```

Gradient Formula
Weight-এর Gradient:

```
1dj_dw = (1/m) * Sum((prediction - y) * x)
```

Bias-এর Gradient:

```
1dj_db = (1/m) * Sum(prediction - y)
```

এখানে,

```
1prediction = w*x + b
```

এবং

```
1prediction - y
```

হলো Error।
আমাদের **Dataset**
ধরি:

```
1X = np.array([1, 3, 4, 6, 7])
```

```
2y = np.array([15, 35, 45, 65, 75])
```

এবং শুরুতে:

```
1w = 0
```

```
2b = 0
```

Step 1: Prediction বের করা
কারণ:

```
1prediction = w*x + b
```

এবং

```
1w = 0
```

```
2b = 0
```

তাই সব Prediction হবে:

```
1[0, 0, 0, 0, 0]
```

Step 2: Error বের করা
Error Formula:

```
1error = prediction - actual
```

তাহলে:

```
10 - 15 = -15
20 - 35 = -35
30 - 45 = -45
40 - 65 = -65
```

```
50 - 75 = -75
```

Error List:

```
1[-15, -35, -45, -65, -75]
```

Step 3: Bias Gradient (dj_db) Calculation
Formula:

```
1dj_db = (1/m) * Sum(error)
```

সব Error যোগ করলে:

```
1-15 + (-35) + (-45) + (-65) + (-75)
```

```
2
```

```
3= -235
```

এবং,

```
1m = 5
```

তাই:

```
1dj_db = -235 / 5
```

```
2
```

```
3dj_db = -47
```

Step 4: Weight Gradient (dj_dw) Calculation

Formula:

```
1dj_dw = (1/m) * Sum(error * x)
```

প্রতিটি Data Point-এর জন্য:

```
1-15 * 1 = -15
```

```
2
```

```
3-35 * 3 = -105
```

```
4
```

```
5-45 * 4 = -180
```

```
6
```

```
7-65 * 6 = -390
```

```
8
```

```
9-75 * 7 = -525
```

এখন সবগুলো যোগ করি:

```
1-15 + (-105) + (-180) + (-390) + (-525)
```

```
2
```

```
3= -1215
```

তাই,

```
1dj_dw = -1215 / 5
```

```
2
```

```
3dj_dw = -243
```

Gradient-এর অর্থ কী**?**

আমরা পেলাম:

```
1dj_dw = -243
2
3dj_db = -47
```

দুইটিই Negative।
এর মানে:

```
1বর্তমান w এবং b খুব ছোট
```

ফলে Model সবক্ষেত্রে Actual Value-এর চেয়ে কম Prediction করছে।
Negative Gradient কী বোঝায়?
যদি Gradient Negative হয়:

```
1Gradient < 0
```

তাহলে Gradient Descent Parameter বাড়ায়।
কারণ Update Rule:

```
1new_parameter = old_parameter - alpha * gradient
```

Negative Gradient বসালে:

```
1new_parameter = old_parameter + কিছু মান
```

ফলে Parameter বৃদ্ধি পায়।
একটি **Numerical Example**
ধরি:

```
1alpha = 0.01
```

এবং:

```
1dj_dw = -243
```

তাহলে:

```
1new_w = 0 - (0.01 * -243)
2
3new_w = 2.43
```

একইভাবে:

```
1new_b = 0 - (0.01 * -47)
2
3new_b = 0.47
```

অর্থাৎ প্রথম Update-এর পর:

```
w = 2.43

b = 0.47
```

Model এখন আগের চেয়ে বড় Prediction দেবে এবং Cost কমবে।

Gradient Calculation-এর পেছনের ধারণা

Gradient মূলত Cost Function-এর Derivative।

Derivative আমাদের বলে:

```
Parameter সামান্য পরিবর্তন করলে Cost কতটা পরিবর্তিত হবে
```

Machine Learning-এ আমরা Derivative-এর ফলাফল ব্যবহার করে Gradient বের করি।

তারপর সেই Gradient ব্যবহার করে Cost কমানোর দিকে এগোই।

Gradient Calculation-এর **Algorithm**

Gradient বের করার সময় আমরা প্রতিটি Data Point-এর জন্য:

```
Prediction বের করি

Error বের করি

Error ব্যবহার করে dj_dw বের করি

Error ব্যবহার করে dj_db বের করি
```

সবশেষে Average নিয়ে Final Gradient পাই।

কেন **Gradient** এত গুরুত্বপূর্ণ**?**

Gradient ছাড়া Gradient Descent কাজ করতে পারে না।

কারণ Gradient-ই বলে দেয়:

```
কোন দিকে যেতে হবে

কতটা পরিবর্তন করতে হবে
```

Gradient Descent শুধু সেই তথ্য ব্যবহার করে Parameter Update করে।

এই মডিউলের মূল ধারণা

Gradient হলো Cost Function-এর ঢাল।
Gradient বলে দেয় Cost কোন দিকে বাড়ছে।
Linear Regression-এ দুইটি Gradient থাকে: dj_dw এবং dj_db।

dj_dw বলে Weight পরিবর্তনে Cost কীভাবে পরিবর্তিত হয়।
dj_db বলে Bias পরিবর্তনে Cost কীভাবে পরিবর্তিত হয়।
Gradient Negative হলে Parameter সাধারণত বাড়ে।
Gradient Positive হলে Parameter সাধারণত কমে।
Gradient Calculation-এর মাধ্যমে Gradient Descent Update Direction নির্ধারণ করে।
Gradient হলো Gradient Descent Algorithm-এর সবচেয়ে গুরুত্বপূর্ণ অংশ।

Applying Gradient Descent to Minimize Cost and Final Prediction

আগের মডিউলগুলোতে আমরা Cost Function, Gradient এবং Gradient Descent-এর মূল ধারণা শিখেছি। আমরা দেখেছি কীভাবে Gradient আমাদের বলে দেয় Cost কোন দিকে বাড়ছে এবং Gradient Descent সেই তথ্য ব্যবহার করে Cost কমানোর দিকে এগিয়ে যায়।

এখন সময় এসেছে পুরো প্রক্রিয়াটি একসাথে দেখার। এই মডিউলে আমরা Gradient Calculation এবং Update Rule ব্যবহার করে সম্পূর্ণ Gradient Descent Algorithm প্রয়োগ করব এবং দেখব কীভাবে Model ধীরে ধীরে সঠিক Weight (w) এবং Bias (b) শিখে ফেলে।

Gradient Descent-এর মূল ধাপগুলো

Gradient Descent প্রতিটি Iteration-এ চারটি কাজ করে:

```
1. Gradient Calculate করে

2. Weight Update করে

3. Bias Update করে

4. নতুন Cost Calculate করে
```

এই প্রক্রিয়াটি বারবার চলতে থাকে যতক্ষণ না Cost সর্বনিম্ন অবস্থায় পৌঁছে।

Gradient Calculation Function

প্রথমে আমরা Gradient Calculate করার Function লিখি।

```
def calculate_gradient(X, y, w, b):
    m = X.shape[0]

    dj_dw = 0.0
    dj_db = 0.0
```

```
6
7    for i in range(m):
8        prediction = w * X[i] + b
9        error = prediction - y[i]
10
11        dj_dw = dj_dw + (error * X[i])
12        dj_db = dj_db + error
13
14    return dj_dw / m, dj_db / m
```

এই **Function** কী করছে?

প্রতিটি Data Point-এর জন্য:

```
1Prediction বের করছে
2
3Error বের করছে
4
5Weight Gradient বের করছে
6
7Bias Gradient বের করছে
```

অর্থাৎ এটি আমাদের বলে:

```
1w কতটুকু পরিবর্তন করা দরকার
2
3b কতটুকু পরিবর্তন করা দরকার
```

Gradient Descent Function

এখন আমরা পুরো Gradient Descent Algorithm লিখব।

```
1def gradient_descent(X, y, w_input, b_input, max_iter, alpha=0.01):
2
3    w = w_input
4    b = b_input
5
6    cost_memo = []
```

```
    iteration = []

    for i in range(max_iter):

        dj_dw, dj_db = calculate_gradient(X, y, w, b)

        w = w - alpha * dj_dw
        b = b - alpha * dj_db

        cost = compute_cost(X, y, w, b)

        cost_memo.append(cost)
        iteration.append(i)

        if i % 100 == 0:
            print(f"Iteration {i}: Cost {cost:.4f}, w: {w:.4f}, b: {b:.4f}")

    return w, b, cost_memo, iteration
```

Update Rule আবার দেখি
Weight Update:

```
w = w - alpha * dj_dw
```

Bias Update:

```
b = b - alpha * dj_db
```

এখানে,

```
alpha = Learning Rate
```

Learning Rate নির্ধারণ করে প্রতিটি Update কত বড় হবে।
এই উদাহরণে:

```
alpha = 0.01
```

Gradient Descent শুরু করা

আমরা শুরুতে Weight এবং Bias শূন্য দিয়ে শুরু করছি।

```
w_final, b_final, cost_memo, iter_list = gradient_descent(
    X,
    y,
    w_input=0,
    b_input=0,
    max_iter=10000,
    alpha=0.01
)
```

শুরুতে:

```
w = 0
```

```
b = 0
```

এবং Cost ছিল:

```
1332.5
```

যা একটি খুব বড় Error নির্দেশ করে।

Iteration-এর সময় কী ঘটে**?**

প্রথম Iteration-এ:

```
Gradient Calculate হয়
```

```

```

```
w Update হয়
```

```

```

```
b Update হয়
```

```

```

```
Cost কিছুটা কমে
```

দ্বিতীয় Iteration-এ:

```
আবার Gradient Calculate হয়
```

```
2
3আবার Update হয়
4
5Cost আরও কমে
```

এভাবে হাজার হাজার Iteration চলতে থাকে।

Cost ধীরে ধীরে কমে

Gradient Descent-এর একটি গুরুত্বপূর্ণ বৈশিষ্ট্য হলো:

```
1প্রতিটি Iteration-এর পর Cost সাধারণত কমতে থাকে
```

উদাহরণস্বরূপ:

```
1Iteration 0
2Cost = 1332.5
3
4Iteration 100
5Cost = 150
6
7Iteration 500
8Cost = 10
9
10Iteration 1000
11Cost = 0.5
12
13Iteration 5000
14Cost = 0.0001
15
16Iteration 10000
17Cost ≈ 0
```

এখানে দেখা যাচ্ছে Model ধীরে ধীরে আরও ভালো Prediction করতে শিখছে।

Gradient-এর মান কেন কমতে থাকে**?**

শুরুতে Gradient-এর মান বড় থাকে।

কারণ:

```
1Model অনেক ভুল করছে
```

তাই Update-ও বড় হয়।
কিন্তু Cost কমতে থাকলে:

```
1Prediction Actual Value-এর কাছাকাছি চলে আসে
```

ফলে Error কমে যায়।
Error কমলে:

```
1dj_dw কমে
```

```
2
```

```
3dj_db কমে
```

অর্থাৎ Gradient ছোট হতে থাকে।
Convergence কী?
এক সময় এমন একটি অবস্থায় পৌঁছানো যায় যেখানে:

```
1Cost আর উল্লেখযোগ্যভাবে কমে না
```

এবং

```
1dj_dw ≈ 0
```

```
2
```

```
3dj_db ≈ 0
```

এই অবস্থাকে বলা হয়:

```
1Convergence
```

অর্থাৎ Model তার Best Parameter-এর কাছাকাছি পৌঁছে গেছে।
Gradient শূন্যের দিকে কেন যায়**?**
Gradient-এর Formula মনে করুন:

```
1dj_dw = (1/m) * Sum((prediction - y) * x)
```

```
2
```

```
3dj_db = (1/m) * Sum(prediction - y)
```

যখন Prediction এবং Actual Value প্রায় সমান হয়ে যায়:

```
prediction - y ≈ 0
```

তখন:

```
dj_dw ≈ 0
```

```
2
```

```
dj_db ≈ 0
```

ফলে Update-ও প্রায় বন্ধ হয়ে যায়।

কেন **Parameter Update** প্রায় থেমে যায়?

Update Formula:

```
w = w - alpha * dj_dw
```

```
2
```

```
b = b - alpha * dj_db
```

যদি:

```
dj_dw ≈ 0
```

```
2
```

```
dj_db ≈ 0
```

হয়, তাহলে:

```
w = w - alpha * 0
```

```
2
```

```
b = b - alpha * 0
```

অর্থাৎ:

```
w এবং b প্রায় অপরিবর্তিত থাকে
```

এই কারণেই বলা হয়:

```
Gradient Descent Minimum Point-এ পৌঁছালে Parameter Update প্রায় বন্ধ হয়ে যায়।
```

Final Prediction

আমাদের Dataset-এর জন্য আদর্শ Line ছিল:

```
prediction = 10*x + 5
```

অর্থাৎ:

```
w = 10
```

```
2
```

```
3b = 5
```

Gradient Descent বহু Iteration-এর পর এই মানগুলোর খুব কাছাকাছি পৌঁছে যায়।
তখন Final Prediction:

```
1x = 1  → 15
```

```
2
```

```
3x = 3  → 35
```

```
4
```

```
5x = 4  → 45
```

```
6
```

```
7x = 6  → 65
```

```
8
```

```
9x = 7  → 75
```

যা Actual Value-এর সাথে মিলে যায়।
Gradient Descent-এর সম্পূর্ণ **Workflow**
পুরো প্রক্রিয়াটি সংক্ষেপে:

```
1Start with random w and b
```

```
2
```

```
3↓
```

```
4
```

```
5Calculate Gradient
```

```
6
```

```
7↓
```

```
8
```

```
9Update w and b
```

```
10
```

```
11↓
```

```
12
```

```
13Calculate New Cost
```

```
14
15↓
16
17Cost কমেছে?
18
19↓
20
21হ্যাঁ → আবার Gradient Calculate করো
22
23↓
24
25না → Minimum Point-এর কাছে পৌঁছে গেছি
26
27↓


28
29Final Model পাওয়া গেল
```

এই মডিউলের মূল ধারণা

Gradient Descent Gradient ব্যবহার করে Weight এবং Bias আপডেট করে।
প্রতিটি Iteration-এ Gradient Calculate করা হয়।
Update Rule ব্যবহার করে নতুন Parameter বের করা হয়।
প্রতিটি Update-এর পরে Cost কমতে থাকে।
Cost কমার সাথে সাথে Gradient-ও ছোট হতে থাকে।
Minimum Point-এর কাছে Gradient প্রায় শূন্য হয়ে যায়।
Gradient শূন্যের কাছাকাছি গেলে Parameter Update প্রায় বন্ধ হয়ে যায়।
এই অবস্থাকে Convergence বলা হয়।
Convergence-এর পর Model Final Prediction দেওয়ার জন্য প্রস্তুত হয়।

Iteration vs Cost Graph

আগের মডিউলে আমরা দেখেছি কীভাবে Gradient Descent ধাপে ধাপে Weight (w) এবং Bias (b) আপডেট করে Cost কমিয়ে আনে। আমরা আরও দেখেছি যে বহু Iteration-এর পর Model সঠিক Parameter শিখে ফেলে এবং ভালো Prediction দিতে সক্ষম হয়।
কিন্তু একটি গুরুত্বপূর্ণ বিষয় হলো, Training-এর সময় Cost আসলেই কমছে কি না তা কীভাবে বুঝব?
এই প্রশ্নের উত্তর পাওয়ার জন্য আমরা **Iteration vs Cost Graph** ব্যবহার করি।
Iteration vs Cost Graph কী?
Iteration vs Cost Graph হলো এমন একটি Graph যেখানে:

```
X-axis = Iteration Number
```

```

```

```
Y-axis = Cost
```

এই Graph আমাদের দেখায়:

```
Gradient Descent-এর প্রতিটি ধাপে Cost কীভাবে পরিবর্তিত হচ্ছে।
```

কেন এই **Graph** গুরুত্বপূর্ণ?
Gradient Descent-এর প্রধান লক্ষ্য:

```
Cost কমানো
```

তাই Training-এর সময় আমরা দেখতে চাই:

```
Cost সত্যিই কমছে কি না
```

যদি Graph-এ দেখা যায় Cost ক্রমাগত কমছে, তাহলে বুঝতে হবে:

```
Model সঠিকভাবে শিখছে
```

Cost History সংরক্ষণ করা
Gradient Descent Function-এর মধ্যে আমরা প্রতিটি Iteration-এর Cost একটি List-এ সংরক্ষণ করেছি।

```
cost_memo.append(cost)
iteration.append(i)
```

এখানে:

```
cost_memo
```

এর মধ্যে প্রতিটি Iteration-এর Cost জমা হচ্ছে।
এবং

```
iteration
```

এর মধ্যে Iteration Number জমা হচ্ছে।

Cost Graph আঁকা

Training শেষ হওয়ার পর আমরা Graph আঁকতে পারি।

```
plt.plot(iter_list[:20], cost_memo[:20])

plt.xlabel("Number of Iterations")
plt.ylabel("Cost")

plt.title("Learning Curve (Cost Reduction)")

plt.show()
```

কেন প্রথম ২০টি **Iteration** দেখানো হয়েছে**?**

Gradient Descent মোট 10000 Iteration চালানো হয়েছে।

```
max_iter = 10000
```

যদি সবগুলো Iteration একসাথে Plot করা হয়, তাহলে শুরুর বড় পরিবর্তনগুলো স্পষ্টভাবে দেখা কঠিন হতে পারে।

তাই উদাহরণে:

```
iter_list[:20]
cost_memo[:20]
```

ব্যবহার করে প্রথম ২০টি Iteration দেখানো হয়েছে।

এতে Cost কীভাবে দ্রুত কমছে তা সহজে বোঝা যায়।

Graph দেখতে কেমন হতে পারে**?**

ধরি প্রথম কয়েকটি Cost এমন:

```
Iteration 0   → Cost = 1332.5

Iteration 1   → Cost = 1090


Iteration 2   → Cost = 890

Iteration 3   → Cost = 720
```

```
Iteration 4   → Cost = 580

Iteration 5   → Cost = 470

Iteration 10  → Cost = 180

Iteration 20  → Cost = 50
```

তাহলে Graph অনেকটা এরকম হবে:

```
Cost

1400 |
     |
1200 | *
     |
1000 |   *
     |
 800 |     *
     |
 600 |         *
     |
 400 |             *
     |
 200 |                 *
     |
   0 |_____________________________

        Iterations
```

এখানে দেখা যাচ্ছে:

```
Iteration বাড়ছে

Cost কমছে
```

এটাই Gradient Descent-এর প্রত্যাশিত আচরণ।

Learning Curve কী?

Iteration vs Cost Graph-কে অনেক সময়:

```
Learning Curve
```

বলা হয়।
কারণ এটি দেখায়:

```
Model সময়ের সাথে কতটা শিখছে।
```

যখন Cost কমে:

```
Model-এর Error কমে
```

অর্থাৎ:

```
Model আরও ভালো Prediction করতে শিখছে।
```

Graph থেকে কী বোঝা যায়**?**
একটি ভালো Learning Curve সাধারণত নিচের দিকে নেমে যায়।

```
Iteration বাড়ে

Cost কমে
```

এটি নির্দেশ করে:

```
Gradient Descent সঠিকভাবে কাজ করছে।
```

Cost দ্রুত কমে তারপর ধীরে কমে কেন**?**
Training-এর শুরুতে Error অনেক বেশি থাকে।
তাই:

```
Gradient বড় হয়

Update বড় হয়
```

ফলে Cost দ্রুত কমতে থাকে।
কিন্তু Training-এর শেষের দিকে:

```
Prediction Actual Value-এর কাছাকাছি চলে আসে
```

তাই:

```
Gradient ছোট হয়

Update ছোট হয়
```

ফলে Cost ধীরে ধীরে কমে।

Convergence Graph-এ কেমন দেখায়?

এক সময় Graph প্রায় সমতল হয়ে যায়।

উদাহরণ:

```
Cost
|
100 |
    |
 50 |
    |
 10 | *
    |   *
  1 |     *
    |       *
0.1 |         *
    |           * * * * * *
    |
    |________________________
          Iterations
```

শেষের অংশে দেখা যাচ্ছে:

```
Cost প্রায় পরিবর্তন হচ্ছে না
```

এটিই Convergence নির্দেশ করে।

Convergence-এর অর্থ

Convergence মানে:

```
Model Minimum Cost-এর খুব কাছাকাছি পৌঁছে গেছে।
```

এই অবস্থায়:

```
dj_dw ≈ 0


dj_db ≈ 0
```

ফলে:

```
w এবং b প্রায় আর পরিবর্তিত হয় না।
```

Final Prediction

Gradient Descent শেষ হওয়ার পর আমরা Final Weight এবং Bias পাই।

```
print(f"Optimized w: {w_final:.4f}, b: {b_final:.4f}")
```

আমাদের Dataset-এর জন্য মানগুলো প্রায়:

```
w = 10


b = 5
```

হয়ে যায়।

নতুন **Data-**এর জন্য **Prediction**

ধরি কারও Experience:

```
x_new = 2
```

Prediction Formula:

```
prediction = w*x + b
```

তাহলে:

```
prediction = 10*2 + 5


prediction = 25
```

Code:

```
x_new = 2


prediction = w_final * x_new + b_final


print(f"Predicted salary for {x_new} years of experience:
{prediction:.2f}k")
```

Output:

```
Predicted salary for 2 years of experience: 25.00k
```

Final Best Fit Line Visualization

Training শেষ হওয়ার পর আমরা Final Prediction Plot করতে পারি।

```
final_preds = make_prediction(
    X,
    y,
    w=w_final,
    b=b_final
)

sns.scatterplot(x=X, y=y)

plt.plot(
    X,
    final_preds,
    color='red',
    label='Best fit line'
)

plt.title(
    f"Final Cost: {compute_cost(X, y, w_final, b_final):0.4e}"
)

plt.legend()

plt.show()
```

এই Plot-এ:

```
Data Point-গুলো Scatter Plot হিসেবে থাকবে

Model-এর শেখা Line লাল রঙে দেখা যাবে
```

যদি Training সফল হয়, তাহলে Line Data Point-গুলোর খুব কাছ দিয়ে যাবে।

পুরো **Module 10-**এর সারাংশ

এই মডিউল সিরিজে আমরা শিখেছি:

Linear Regression Model তৈরি করা।
Prediction Function ব্যবহার করে Output বের করা।
Cost Function ব্যবহার করে Error মাপা।
Cost Function Visualization করা।
Cost Minimization-এর ধারণা বোঝা।
Gradient এবং Gradient Descent বোঝা।
Gradient Calculation করা।
Gradient Descent ব্যবহার করে Weight ও Bias আপডেট করা।
Final Best Fit Line বের করা।
Iteration vs Cost Graph ব্যবহার করে Training Progress পর্যবেক্ষণ করা।

এই মডিউলের মূল ধারণা

Iteration vs Cost Graph Training Progress দেখায়।
X-axis-এ Iteration এবং Y-axis-এ Cost থাকে।
Cost কমতে থাকলে Model শিখছে।
এই Graph-কে Learning Curve-ও বলা হয়।
Training-এর শুরুতে Cost দ্রুত কমে।
পরে ধীরে ধীরে কমে এবং স্থির হয়ে যায়।
Graph সমতল হয়ে গেলে Convergence হয়েছে বোঝা যায়।
Convergence-এর সময় Gradient প্রায় শূন্য হয়ে যায়।
Final Weight ও Bias ব্যবহার করে নতুন Data-এর জন্য Prediction করা যায়।

এর মাধ্যমে আমরা Scratch থেকে সম্পূর্ণ Linear Regression এবং Gradient Descent-এর Workflow সফলভাবে সম্পন্ন করলাম।